Order Book Engine

Moteur de carnet d'ordres haute performance
et agents de trading par apprentissage par renforcement

Projet de Fin d'Année (PFA)

Yani Haouchine

5 mai 2026

ENSEIRB-MATMECA

Table des matières

1 Introduction **3**

2 Besoins du client **3**

3 Cahier des charges **3**
3.1 Exigences fonctionnelles . . . 3
3.2 Exigences non fonctionnelles . . . 3
3.3 Périmètre . . . 4

4 Organisation du projet **4**
4.1 Équipe et rôles . . . 4
4.2 Méthodologie . . . 4
4.3 Outils . . . 4
4.4 Arborescence . . . 4

5 Diagramme de Gantt **5**

6 Techniques mises en œuvre **5**
6.1 Moteur de carnet d'ordres . . . 5
6.1.1 Structures de données . . . 5
6.1.2 Algorithme de matching . . . 5
6.2 Données de marché temps réel . . . 5
6.3 Agents de trading . . . 5
6.3.1 Stratégies par règles . . . 5
6.3.2 Apprentissage par renforcement . . . 6
6.4 Infrastructure d'entraînement . . . 6
6.5 Tableau de bord . . . 6

7 Expérimentations **6**
7.1 Benchmarks du moteur . . . 6
7.2 Entraînements RL . . . 7
7.3 Validation manuelle . . . 7

8 Résultats et discussion **7**
8.1 Performance du moteur . . . 7
8.2 Apprentissage des agents RL . . . 8
8.2.1 A2C . . . 8
8.2.2 PPO . . . 9
8.2.3 Dreamer v3 . . . 10
8.3 Comparaison des stratégies . . . 10
8.4 Discussion . . . 10

9 Manuel d'utilisation **11**
9.1 Pré-requis . . . 11
9.2 Compilation . . . 11
9.3 Lancement du moteur et des données live . . . 11
9.4 Lancement d'un agent . . . 11
9.5 Entraînement des agents RL . . . 12
9.6 Tableau de bord . . . 12
9.7 Configuration Binance testnet . . . 12

10 Maintenance **12**
10.1 Mise à jour des dépendances 12
10.2 Ajout d'une nouvelle stratégie 12
10.3 Ré-entraînement périodique 12
10.4 Surveillance en production 12
10.5 Tests de non-régression 13

11 Conclusion **13**

1 Introduction

Ce rapport présente le projet *Order Book Engine*, un moteur de carnet d'ordres haute performance écrit en C++20, complété par une suite d'agents de trading développés en Python. Le projet couvre à la fois l'aspect **infrastructure de marché** (matching engine, intégration de données temps réel) et l'aspect **stratégie de trading** (agents par règles et agents par apprentissage par renforcement).

2 Besoins du client

Le client souhaite disposer d'une plateforme permettant :

— de **simuler un carnet d'ordres** fidèle aux conventions des marchés financiers (priorité prix-temps, plusieurs types d'ordres) ;

— de **récupérer en temps réel** les données d'un marché crypto (Binance) pour alimenter la simulation ;

— d'**exécuter des stratégies de trading algorithmiques** sur ce flux temps réel, depuis les stratégies classiques (mean reversion, RSI, Bollinger) jusqu'aux agents d'*apprentissage par renforcement* (DQN, A2C, PPO, Dreamer v3) ;

— de **visualiser le marché et les performances** via un tableau de bord interactif ;

— de mesurer la **performance brute** du moteur (latence, débit) afin d'évaluer sa pertinence dans un contexte applicatif réaliste.

3 Cahier des charges

3.1 Exigences fonctionnelles

— **F1 – Moteur de matching** : gestion des ordres d'achat/vente avec priorité prix-temps (FIFO intra-niveau).

— **F2 – Types d'ordres supportés** : `GoodTillCancel`, `Market`, `ImmediateOrCancel`, `FillOrKill`, `GoodForDay`.

— **F3 – Données de marché temps réel** : intégration de l'API Binance (REST) pour récupérer le carnet d'ordres et les agrégats de transactions.

— **F4 – Agents de trading** : architecture modulaire permettant de brancher facilement plusieurs stratégies (règles, RL).

— **F5 – Tableau de bord** : visualisation du marché, des positions, du PnL et des métriques d'apprentissage.

— **F6 – Persistance** : sauvegarde des modèles entraînés et des métriques d'entraînement (CSV/PNG).

3.2 Exigences non fonctionnelles

— **NF1 – Performance** : insertion d'ordres $\geq 100\,\text{k}$ ops/s, latence d'opération de l'ordre de la microseconde.

— **NF2 – Portabilité** : compilation sous Linux / macOS / Windows (CMake + C++20).

— **NF3 – Maintenabilité** : séparation claire entre cœur de matching, accès aux données de marché et logique d'agent.

— **NF4 – Reproductibilité** : scripts d'entraînement déterministes (graines aléatoires, checkpoints, logs).

3.3 Périmètre

Inclus. Moteur, ingestion données Binance, agents règles et RL, dashboard, benchmarks.

Exclus. Trading en argent réel, gestion multi-comptes, conformité réglementaire, optimisation FPGA.

4 Organisation du projet

4.1 Équipe et rôles

— Développement moteur C++ : *à compléter*
— Intégration données de marché : *à compléter*
— Agents de trading et RL : *à compléter*
— Tableau de bord : *à compléter*

4.2 Méthodologie

Le projet a été conduit de manière itérative : une première itération a livré le moteur de matching et ses tests, puis l'ingestion de données réelles, puis successivement les agents règles, RL et le dashboard. Chaque itération s'est appuyée sur des benchmarks pour valider la non-régression de la performance.

4.3 Outils

— **Versionnage** : Git (branches *feature* + intégration sur `master`).
— **Build** : CMake 3.10+, GCC/Clang.
— **Dépendances** : `libcurl 8.4.0`, `nlohmann/json 3.11.3` (téléchargées via `FetchContent`).
— **Python** : PyTorch (CUDA quand disponible), NumPy, Matplotlib.
— **Tableau de bord** : HTML/CSS/JS + serveur Python.

4.4 Arborescence

```
ein8-proj2-pfa-27707/
|-- CMakeLists.txt
|-- include/                # Coeur, OrderBook, MarketData, LiveMarketData
|-- src/                    # Implémentations C++
|   |-- OrderBook/
|   `-- LiveMarketData/
|-- tests/                  # Tests fonctionnels et benchmarks
|-- agents/                 # Agents Python
|   |-- main.py             # Point d'entrée trading agent
|   |-- strategies/         # Mean reversion, RSI, Bollinger,
|   |                       #   DQN, A2C, PPO, Dreamer
|   |-- exchange/           # Client Binance testnet
|   |-- train_dqn.py
|   |-- train_a2c.py
|   `-- train_dreamer.py
|-- dashboard/              # Tableau de bord web
|-- data/                   # Données de marché (CSV)
|-- models/                 # Modèles RL entraînés (.pth)
`-- README.md
```

Listing 1 – Structure du dépôt (extrait)

5 Diagramme de Gantt

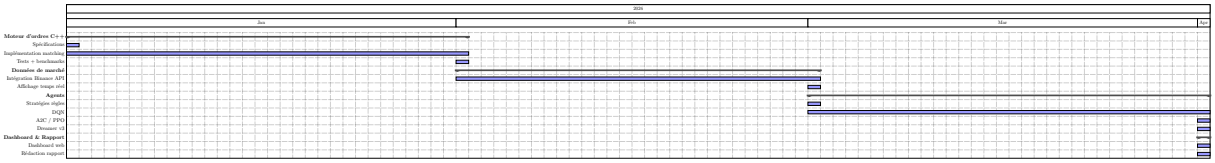


FIGURE 1 – Diagramme de Gantt prévisionnel du projet

6 Techniques mises en œuvre

6.1 Moteur de carnet d'ordres

6.1.1 Structures de données

Le carnet d'ordres est représenté par deux `std::map` distinctes pour les bids et les asks. Cette séparation permet deux ordres de tri opposés (descendant pour les achats, ascendant pour les ventes) et un accès $O(\log n)$ au meilleur prix. Chaque niveau de prix contient une liste FIFO d'ordres (`std::list<OrderPointer>`).

Une table de hachage `std::unordered_map<OrderId, OrderEntry>` stocke en plus, pour chaque ordre, un itérateur sur sa position dans la liste : cela permet l'**annulation en** $O(1)$ d'un ordre par identifiant.

Complexités obtenues

Opération	Complexité	Justification
Ajout ordre	$O(\log n)$	insertion dans la map des niveaux
Annulation	$O(1)$	itérateur stocké dans `OrderEntry`
Modification	$O(\log n)$	cancel + insert
Matching	$O(k \log n)$	k trades exécutés

6.1.2 Algorithme de matching

Les ordres se matchent dès que le prix d'un buy est $\geq$ au meilleur ask, ou symétriquement. La priorité est **prix-temps** :

1. meilleur prix d'abord ;
2. au sein d'un niveau, ordre d'arrivée (FIFO) ;
3. exécutions partielles autorisées (sauf `FillOrKill`).

Les ordres `Market` sont convertis en ordres limites agressifs (prix extrême) afin de **réutiliser le code de matching** existant et d'éviter une branche dédiée.

6.2 Données de marché temps réel

L'exécutable `LiveMarketData` interroge l'API publique Binance via `libcurl`, parse la réponse JSON avec `nlohmann/json`, puis injecte le carnet dans le moteur. Un `Display` ANSI affiche en continu le *top of book*, la SMA et les ordres agents.

6.3 Agents de trading

6.3.1 Stratégies par règles

— **Mean Reversion** : achète quand le prix est $-2\,\%$ sous la SMA, vend si $+2\,\%$ au-dessus.

— **RSI** : RSI sur 14 périodes, achat si < 30, vente si > 70.
— **Bollinger Bands** : achat/vente quand le prix sort des bandes à $\pm 2\sigma$.

6.3.2 Apprentissage par renforcement

DQN (Deep Q-Network). Réseau *dueling* avec activations SiLU et LayerNorm, **Double DQN**, *replay buffer* de 200 000 transitions, perte Smooth-L1, gradient clipping. L'espace d'état est de dimension 6 (imbalance, spread, déviation, momentum, inventaire, volatilité réalisée). Trois actions : BUY, SELL, HOLD. Hyperparamètres : $\gamma = 0.995$, lr $= 3 \times 10^{-4}$, batch 256, ε décroissant $1.0 \to 0.01$.

A2C (Advantage Actor-Critic). Acteur et critique partageant un tronc commun, mise à jour synchrone par avantage estimé. Variante permettant de comparer une approche *policy-based* au DQN *value-based*.

PPO (Proximal Policy Optimization). *Clipped surrogate objective* pour stabiliser les mises à jour de politique. L'apprentissage est plus stable que A2C grâce à la borne sur le ratio $\pi_\theta / \pi_{\theta_{\text{old}}}$.

Dreamer v3 (model-based). Architecture la plus avancée du projet : un **World Model** composé d'un encodeur, d'un *Recurrent State-Space Model* (RSSM) à latents catégoriels (32×32) et d'un GRU, puis un **acteur-critique entraîné en imagination** sur les rollouts du world model. Utilise `symlog`/`symexp` pour la stabilité numérique des récompenses à grande dynamique.

6.4 Infrastructure d'entraînement

— Détection automatique de CUDA, repli sur CPU.
— Chargement *streaming* des CSV Binance (jusqu'à 3 Go) pour ne pas saturer la RAM.
— Sauvegarde périodique des poids (`models/*.pth`).
— Métriques temps réel : loss, reward, PnL, inventaire, ratio actions, Sharpe approché, drawdown, mémoire GPU.

6.5 Tableau de bord

Front-end statique HTML/CSS/JS servi par un petit serveur Python ; lit les fichiers JSON `data/market_state.json` et `data/agent_orders.json` produits par le moteur et l'agent.

7 Expérimentations

7.1 Benchmarks du moteur

L'exécutable `OrderBookBenchmark` mesure la latence et le débit des opérations clés sur un volume de plusieurs millions d'ordres synthétiques.

Opération	Complexité	Débit mesuré
Ajout d'ordre	$O(\log n)$	≈ 400 k ops/s
Annulation	$O(1)$	≈ 2 M ops/s
Modification	$O(\log n)$	≈ 270 k ops/s
Matching	$O(k \log n)$	≈ 350 k matches/s
Snapshot `GetOrderInfo`	$O(m)$	≈ 500 k snapshots/s

TABLE 1 – Performances mesurées du moteur (latence 2–4 μs).

7.2 Entraînements RL

Les agents RL ont été entraînés sur des données *aggTrades* Binance :

— DQN : `BTCUSDT-aggTrades-2026-03.csv` (lecture en streaming, $\sim$ 12 M lignes/époque).

— A2C, PPO, Dreamer v3 : `SOLUSDT-aggTrades-2026-02.csv`.

Un protocole identique a été appliqué : *warm-up* de 2000 lignes, log toutes les 2000 itérations, sauvegarde du modèle à chaque époque, suivi des actions (*hold/buy/sell*), du PnL *mark-to-market* et du *Sharpe ratio* glissant.

7.3 Validation manuelle

Des scripts `test_dqn_manual.py` et `test_sma_manual.py` vérifient que :

— BUY est sélectionné dans les contextes attendus ;

— SELL est bloqué quand l'inventaire est nul (*inventory guard*) ;

— la SMA est calculée correctement sur la fenêtre glissante.

8 Résultats et discussion

8.1 Performance du moteur

Les chiffres du tableau 1 montrent que le moteur atteint des débits de l'ordre de la **centaine de milliers d'opérations par seconde** en insertion et plusieurs **millions** en annulation. La latence reste dans la fourchette 2–4 μs, compatible avec un usage en trading algorithmique côté *buy-side* (hors HFT extrême).

8.2 Apprentissage des agents RL

8.2.1 A2C

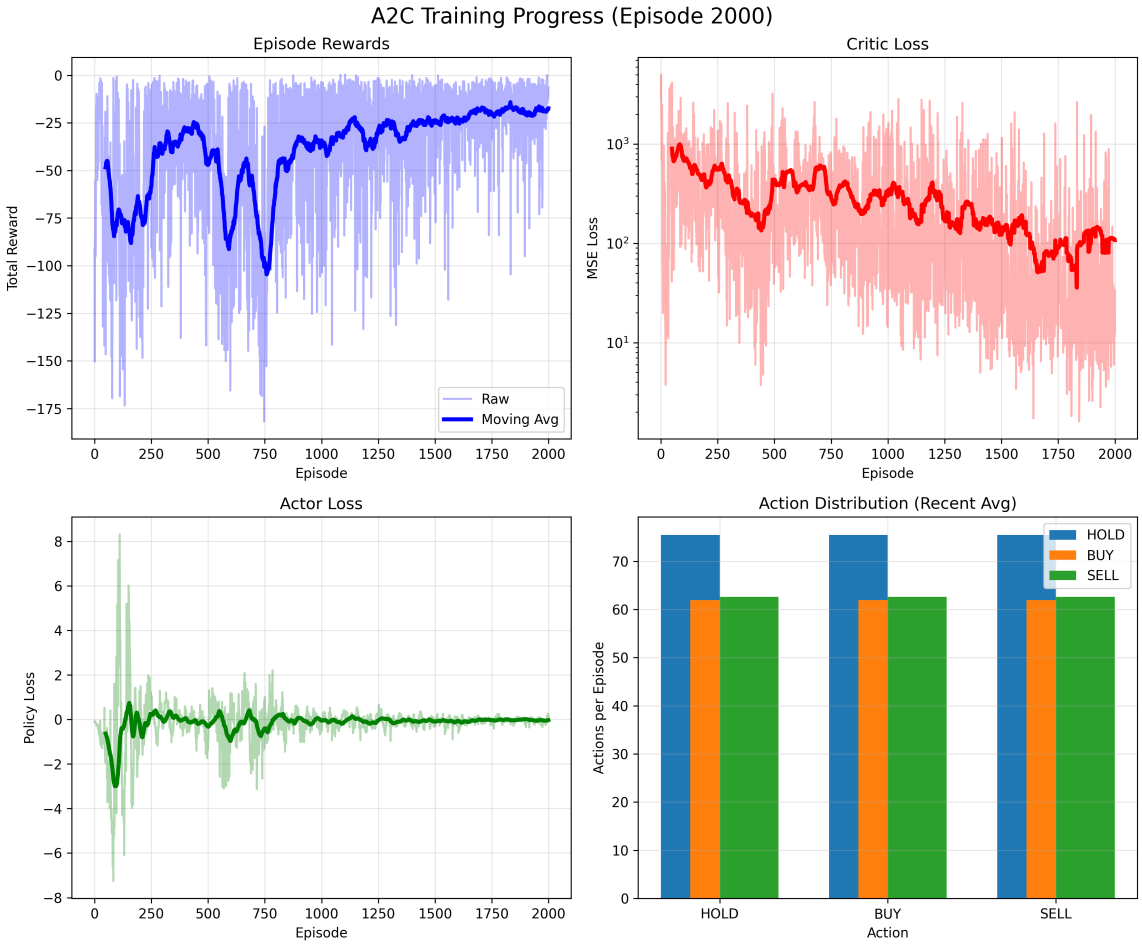


FIGURE 2 – Courbes d'apprentissage de l'agent A2C (loss, récompense, PnL).

8.2.2 PPO

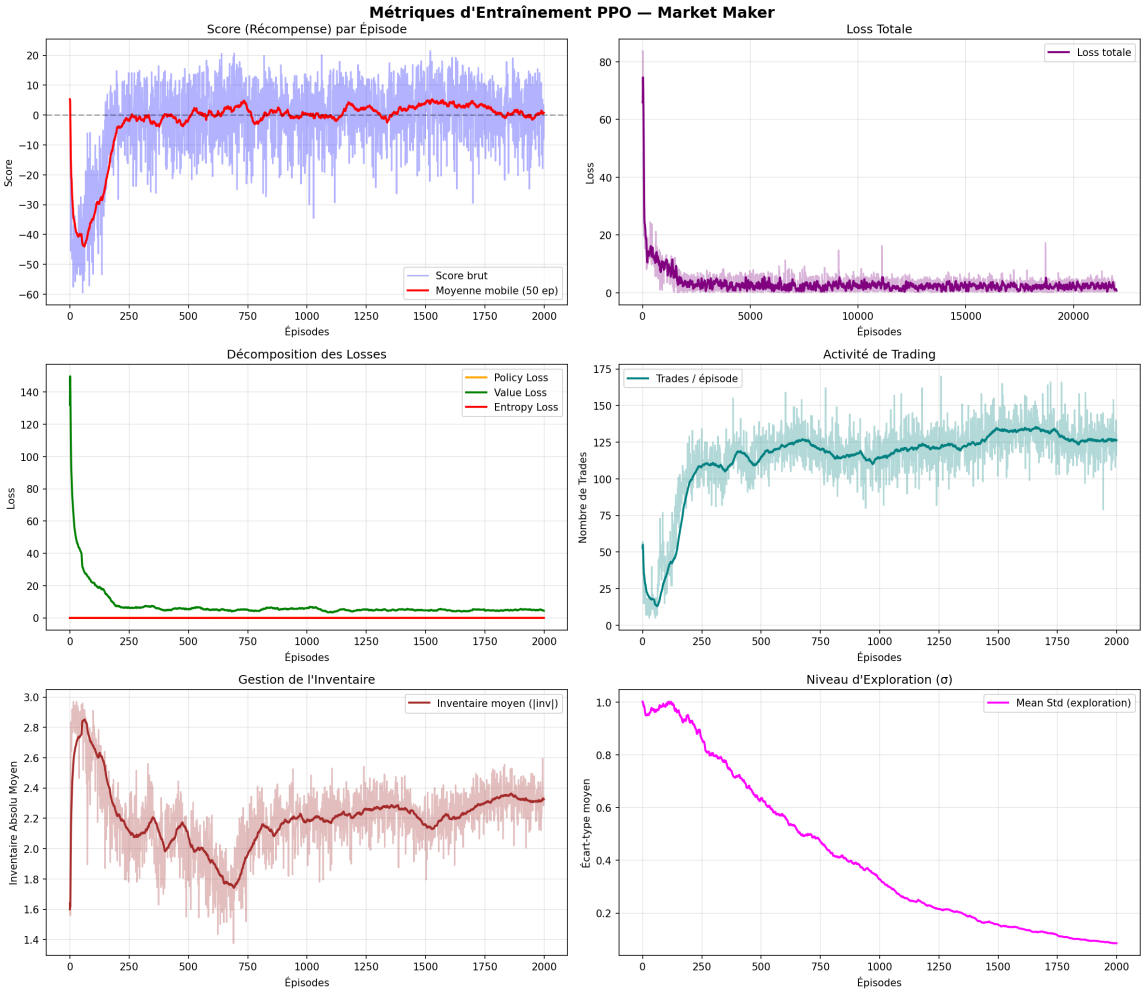


FIGURE 3 – Courbe d'apprentissage de l'agent PPO.

8.2.3 Dreamer v3

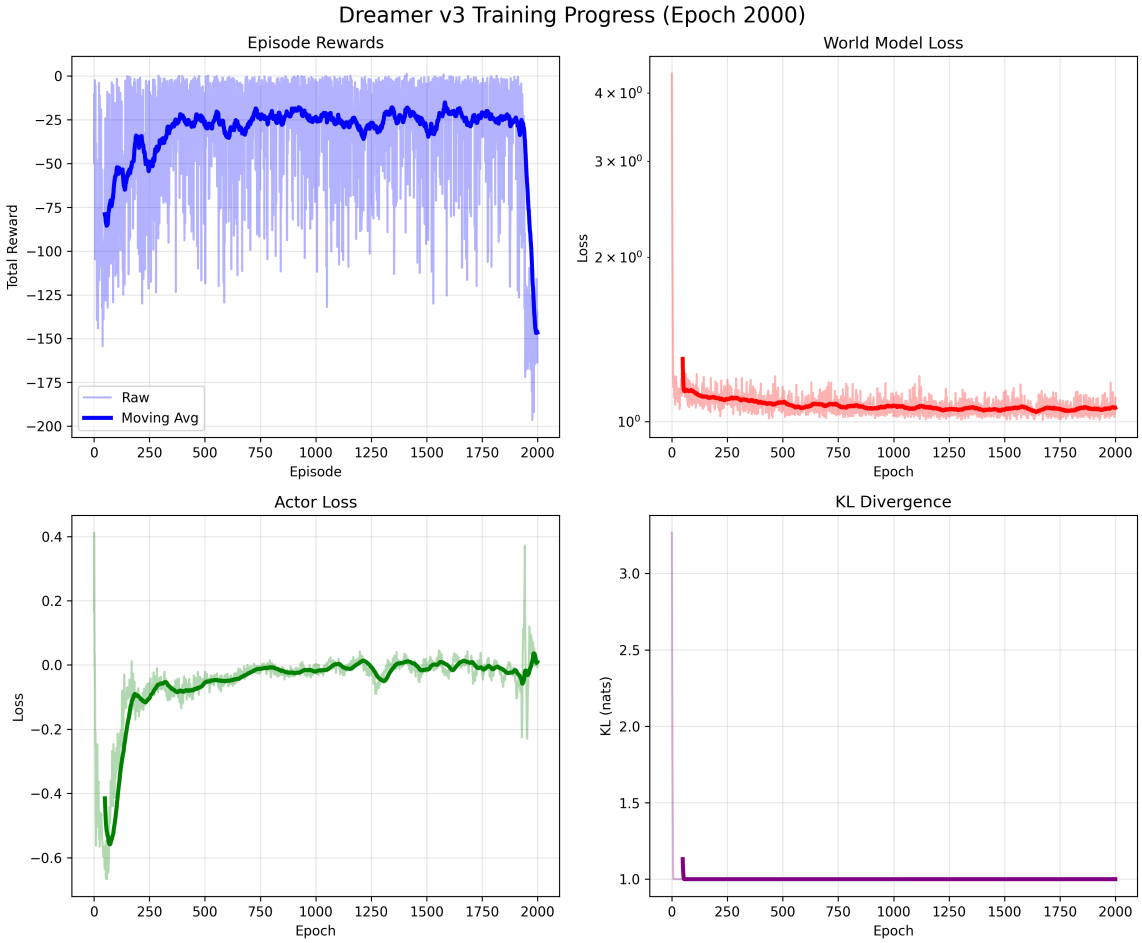


FIGURE 4 – Courbes d'apprentissage de Dreamer v3 : loss du world model, loss acteur/critique, récompense et PnL.

8.3 Comparaison des stratégies

Stratégie	**PnL final**	**Sharpe**	**Drawdown max**	**Remarques**
Mean Reversion	—	—	—	baseline règles
RSI	—	—	—	baseline règles
Bollinger	—	—	—	baseline règles
DQN	—	—	—	value-based
A2C	—	—	—	policy-based
PPO	—	—	—	policy-based stable
Dreamer v3	—	—	—	model-based

TABLE 2 – Comparaison des stratégies sur jeu de test commun.

8.4 Discussion

Ce qui fonctionne. Le moteur tient ses objectifs de performance, l'intégration Binance est stable, les stratégies par règles donnent une *baseline* interprétable et les agents RL convergent

visiblement (figures 2, 3, 4).

Limites observées. Les agents RL restent sensibles à la qualité du *reward shaping* et à la non-stationnarité des marchés crypto.

Pistes d'amélioration.

— Enrichir l'espace d'état (carnet multi-niveaux, *order flow imbalance*).

— Tester un *reward* à risque ajusté (Sharpe différentiel).

— Intégrer un *walk-forward validation* plus systématique.

9 Manuel d'utilisation

9.1 Pré-requis

— CMake $\geq$ 3.10, compilateur C++20 (GCC 10+, Clang 10+ ou MSVC 2019+).

— Python 3.10+, PyTorch (CUDA optionnel mais recommandé pour le DQN).

— Connexion Internet (API Binance, téléchargement des dépendances `libcurl` / `nlohmann/json`).

9.2 Compilation

```
# Première fois (configuration initiale)
mkdir build && cd build
cmake ..
cmake --build . --config Release -j$(nproc)

# Builds suivants
./build.sh
```

9.3 Lancement du moteur et des données live

```
# Tests + benchmarks
./build/OrderBookTests

# Affichage temps réel d'un symbole
./build/LiveMarketData SOLUSDT 1 20
# [SYMBOLE] [RAFRAICHISSEMENT_S] [PROFONDEUR]
```

9.4 Lancement d'un agent

```
# Terminal 1 : flux de marché C++
./run_market.sh

# Terminal 2 : agent Python (stratégie au choix)
STRATEGY=mean_reversion python3 -m agents.main
STRATEGY=rsi            python3 -m agents.main
STRATEGY=bollinger      python3 -m agents.main
```

9.5 Entraînement des agents RL

```
# DQN (variante normale)
python3 -m agents.train_dqn --variant normal \
    --csv BTCUSDT-aggTrades-2026-03.csv \
    --epochs 1 --rows-per-epoch 500000 \
    --warmup-rows 2000 --log-every 2000

# A2C
python3 -m agents.train_a2c

# Dreamer v3 (avec affichage live)
python3 run_dreamer.py
python3 run_dreamer.py --fast      # configuration CPU
```

9.6 Tableau de bord

```
./run_dashboard.sh
# Puis ouvrir http://localhost:<port>/ dans le navigateur
```

9.7 Configuration Binance testnet

L'agent live utilise les clés API Binance testnet définies dans `agents/exchange/config.py` (à créer à partir de `config.example.py`). Les clés peuvent être obtenues sur https://testnet.binance.vision/.

10 Maintenance

10.1 Mise à jour des dépendances

— **C++** : les versions de `libcurl` et `nlohmann/json` sont fixées via `FetchContent` dans `CMakeLists.txt`. Mettre à jour les tags pour récupérer une nouvelle version, puis rebuilder à blanc : `rm -rf build && ./build.sh`.

— **Python** : `pip install -U torch numpy matplotlib pandas requests`.

10.2 Ajout d'une nouvelle stratégie

1. Créer `agents/strategies/ma_strategie.py` implémentant l'interface `BaseStrategy`.
2. L'enregistrer dans `agents/main.py` (branche `if strategy_name == ...`).
3. Tester via `STRATEGY=ma_strategie python3 -m agents.main`.

10.3 Ré-entraînement périodique

Les marchés évoluent : il est recommandé de **ré-entraîner les agents RL** sur des données récentes au moins une fois par mois, en vérifiant la non-régression du PnL sur un jeu de validation *out-of-sample*.

10.4 Surveillance en production

— Lire les logs `dreamer_train.log` et les CSV `dqn_metrics/*.csv`.

— Surveiller la latence de l'API Binance.

— Conserver les checkpoints `models/*.pth` versionnés.

10.5 Tests de non-régression

```
./build/OrderBookTests          # tests fonctionnels et perf C++
python3 test_dqn_manual.py      # validation actions DQN
python3 test_sma_manual.py      # validation SMA
```

11 Conclusion

Ce projet démontre la faisabilité d'une plateforme de trading complète associant un **moteur de carnet d'ordres en C++20 performant** ($O(\log n)$ insertion, $O(1)$ annulation, latences microsecondes), une **ingestion temps réel** de données Binance, une **collection d'agents de trading** couvrant à la fois les heuristiques classiques (mean reversion, RSI, Bollinger) et les méthodes d'apprentissage par renforcement modernes (DQN, A2C, PPO, Dreamer v3), et enfin un **tableau de bord** pour la visualisation.

Les expérimentations ont validé les exigences de performance du moteur, ainsi que la convergence des agents sur les jeux de données utilisés. Les pistes d'extension naturelles concernent l'enrichissement de l'espace d'état (*order flow imbalance*, profondeur multi-niveaux), l'introduction de *rewards* à risque ajusté, et un protocole de *walk-forward validation* systématique pour mieux capturer la non-stationnarité des marchés crypto.